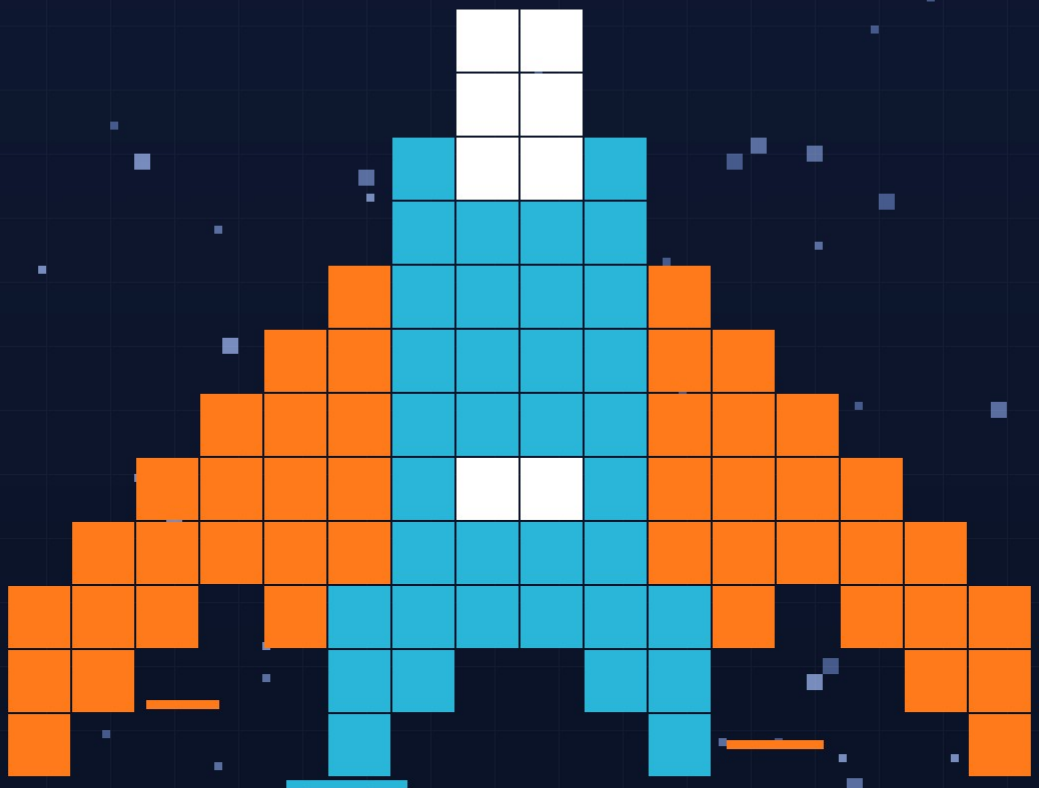


MANUAL DE REINGENIERÍA · INVESTIGACIÓN · ESPECIFICACIÓN

B-WINGS

Manual de reingeniería, investigación y especificación técnica

Reconstrucción fiel de la versión Famicom/NES de 1986

VERSIÓN 0.3 · EDICIÓN UNIFICADA · OCTUBRE DE 2026

Documento de referencia vivo · investigación en curso

PRELIMINARES

Antes de empezar

> **Objetivo del documento.** Documentar el juego de forma comprensible para cualquier lector y establecer una base verificable para una futura reconstrucción independiente. Este manual es la «biblia» del proyecto: reúne cada detalle del funcionamiento de B-Wings junto con su nivel de evidencia, de modo que un programador pueda reconstruir el comportamiento en el lenguaje o motor que prefiera.

Ficha del documento

Campo	Valor
Título	B-Wings — Manual de reingeniería, investigación y especificación técnica
Juego estudiado	B-Wings (Data East), versión Famicom/NES de 1986
Versión del manual	0.3 — edición unificada, formato libro
Fuentes unificadas	Manual v0.2 (investigación), Guía técnica de arquitectura NES y reimplementación limpia en Godot, y los últimos hallazgos sobre WINGs, Morph y Laser Wall
Meta del proyecto	Ejecutable de escritorio con tres modos: Original, Corregido/Enhanced y Rebuild extendido
Motor previsto	Godot 4 (diseño de referencia); la elección definitiva se confirma al cerrar la especificación
Estado de la investigación	En curso — ver el mapa de conocimiento (capítulo 25)
Idioma	Español

Cómo leer este manual

Cada afirmación lleva implícita una clasificación de evidencia (**HECHO**, **INFERENCIA**, **HIPÓTESIS**, **DECISIÓN** o **PENDIENTE**; capítulo 3). Las direcciones de memoria se escriben con el prefijo `$` y en tipografía de código, por ejemplo `$00CF`. Los recuadros de color resaltan reglas, principios, advertencias y fuentes externas. Las secciones marcadas como pendientes son huecos reales de la investigación, no omisiones del texto.

> **Nota.** Los recuadros de este libro siguen un código de color: azul para aclaraciones y notas, verde para reglas y principios, ámbar para advertencias y pendientes críticos, gris para citas o fuentes externas.

HECHO	INFERENCIA	HIPÓTESIS	DECISIÓN	PENDIENTE
Comprobado	Derivado de hechos	A poner a prueba	Elección del proyecto	Sin evidencia aún

Los marcos grises con la leyenda «Espacio reservado para captura» indican dónde colocar fotografías o capturas del juego original. Cada marco lleva su epígrafe y la referencia de qué parte del juego se está explicando.

Cada vez que el equipo o su asistente de investigación descubra algo nuevo, se incorpora al capítulo correspondiente y se actualiza el mapa de conocimiento y el registro de cambios del Apéndice C.

CONTENIDO

Índice

Parte I · Fundamentos 4
1. Propósito y filosofía 5
2. Alcance y criterios legales 7
3. Método de investigación 9

Parte II · Hardware y cartucho 12
4. Arquitectura de hardware NES 13
5. Identificación del juego y de la ROM 15

Parte III · El juego 17
6. Modelo de gameplay 18
7. Sistema de WINGs 21
8. Sistema Morph 26
9. Estructuras 28
10. Enemigos 30
11. Etapas 32
12. Bosses 34

Parte IV · Ingeniería inversa de la ROM 36
13. Arquitectura interna observada 37
14. Mapa de RAM 40
15. Scrolling 42
16. Proyectiles y entidades 44
17. Slowdown 46
18. Glitches y colisiones anómalas 49
19. Agenda de investigación en la ROM 52

Parte V · Reimplementación en PC 54
20. Qué conservar y qué mejorar 55
21. Arquitectura propuesta 57
22. Validación y pruebas de regresión 60
23. Plan de desarrollo 62
24. Documentación, distribución y próximos pasos 64

Parte VI · Estado y cierre 66
25. Mapa de conocimiento 67
26. Conclusión 69

Apéndices 71
Apéndice A · Glosario 72
Apéndice B · Fuentes y referencias 74
Apéndice C · Reconciliación y registro de cambios 76
Apéndice D · Plantillas de registro 78

PARTE

Fundamentos

Propósito, alcance legal y método de trabajo.

Capítulos de esta parte

- 1. Propósito y filosofía
- 2. Alcance y criterios legales
- 3. Método de investigación

CAPÍTULO

01

Propósito y filosofía

Por qué existe el proyecto, qué significa fidelidad y cómo se organizan los tres modos de juego del ejecutable.

En este capítulo

- Orden de prioridad
- 1.1 Fidelidad
- 1.2 Modos de juego del ejecutable

El proyecto no busca hacer un remake moderno ni un simple port. La pregunta central es: ¿cómo sería este mismo B-Wings si se mantuvieran sus reglas, niveles, armas, enemigos, dificultad, estética y ritmo, pero se eliminaran las limitaciones técnicas que no forman parte del diseño?

De esa pregunta salen dos productos complementarios: un **manual** que documenta cada comportamiento con evidencia (este documento) y un **motor** para PC que lo reproduce y, opcionalmente, lo amplía.

Orden de prioridad

1. Fidelidad al original.
2. Corrección de limitaciones técnicas demostradas.
3. Mejoras opcionales.
4. Arquitectura simple para iterar rápidamente.

1.1 Fidelidad

Fidelidad significa reproducir no sólo gráficos, sino también movimiento, aceleración, cadencia de disparo, límites de proyectiles, estados de daño, comportamiento enemigo, scrolling, colisiones, progresión y timing.

1.2 Modos de juego del ejecutable

En lugar de mantener «versiones» separadas del programa, el ejecutable usa perfiles de parámetros cargados desde archivos de datos y activados por configuración.

Modo	Objetivo	Qué cambia
Original	Reproducir el comportamiento medido del Famicom.	Nada: reglas, límites, slowdown y glitches confirmados se conservan.
Corregido / Enhanced	Conservar las reglas y eliminar limitaciones técnicas demostradas.	Correcciones de glitches, estabilidad de timing (por ejemplo 60 FPS nativos), ausencia de límite artificial de sprites. Las correcciones se activan sólo de forma explícita.
Rebuild / Extendido	Plataforma para contenido nuevo sin perder la base del original.	Más niveles, más enemigos, más disparos en pantalla y niveles creados por la comunidad mediante datos.

> **Regla.** Enhanced no significa hacerlo más fácil. Una mejora técnica no debe cambiar las reglas de diseño. Los cambios de contenido pertenecen exclusivamente al modo Rebuild.

> **Decisión confirmada.** El manual v0.2 sólo definía los modos Original y Enhanced; la guía técnica proponía «original, corregido, extendido». El proyecto confirmó los tres modos: Original, Corregido/Enhanced y Rebuild/Extendido.

CAPÍTULO

02

Alcance y criterios legales

Los límites legales del estudio y la estrategia de reconstrucción independiente (clean room) con assets propios.

En este capítulo

- 2.1 Principios de trabajo
- 2.2 Estrategia clean-room

Este manual describe cómo analizar hardware y software de la NES con fines de estudio, interoperabilidad y diseño de motores propios. No contiene código desensamblado, tablas de datos extraídas de juegos concretos, gráficos, música ni mapas de niveles de ningún título comercial. Registra direcciones de memoria, comportamientos observados y especificaciones propias.

2.1 Principios de trabajo

- **Reimplementación limpia (clean room).** Quien documenta el comportamiento no debe ser quien escribe el código final, o al menos debe trabajar sólo con especificaciones de hardware y observaciones propias.
- **Assets propios.** Todo sprite, sonido y nivel de la versión distribuida debe ser creado desde cero o contar con licencia explícita.
- **ROM legal.** Cualquier prueba con la ROM original debe hacerse con una copia que el usuario posea legalmente, sin distribuirla.

2.2 Estrategia clean-room

La estrategia recomendada es la reconstrucción independiente: documentar comportamiento, estados y tablas, y escribir una implementación nueva que produzca esos resultados. No se debe trasladar código propietario de la ROM.

> **Consecuencia.** La protección del cartucho se estudia para entender la ROM, pero no se copia como gameplay en PC. Lo mismo vale para cualquier rutina interna: se documenta qué hace, no cómo está escrita.

> **Advertencia sobre las imágenes.** Las capturas del juego original sirven para estudio y para orientar al lector en la copia de trabajo, pero son material con derechos de Data East. En una edición pública deben reemplazarse por ilustraciones propias.

CAPÍTULO

03

Método de investigación

El método con el que se clasifica cada afirmación, se mide cada comportamiento y se separa una limitación de un bug o de una decisión de diseño.

En este capítulo

- 3.1 Clasificación de las afirmaciones
- 3.2 Cadena de análisis
- 3.3 Procedimiento de observación
- 3.4 Plantilla de registro
- 3.5 Limitación, bug o decisión de diseño

Toda afirmación del manual se clasifica en una de cinco categorías de evidencia. Esto evita convertir suposiciones en hechos y permite saber en cualquier momento qué parte de la especificación está firme y cuál no.

3.1 Clasificación de las afirmaciones

Categoría	Significado	Ejemplo en este manual
HECHO / OBSERVADO	Comprobado en la ROM o en emulación, o corroborado por fuentes independientes.	`$00CF` funciona como selector de WING.
INFERENCIA	Conclusión razonable derivada de hechos, pero no observada directamente.	`$00A0/$00A1` participan en el estado que permite el glitch del Laser Wall.
HIPÓTESIS	Explicación posible que todavía debe ponerse a prueba.	Las colisiones son la causa principal del slowdown.
DECISIÓN	Elección de diseño del proyecto, no un descubrimiento.	Enhanced debe eliminar una limitación técnica comprobada.
PENDIENTE	Aún no investigado o sin evidencia suficiente.	Causa exacta del glitch del Laser Wall.

Además de la categoría, el mapa de RAM y el mapa de conocimiento usan un nivel de confianza: **Alto**, **Medio/Alto**, **Medio**, **Bajo/Medio**, **Bajo** o **Pendiente**.

3.2 Cadena de análisis

Cada problema detectado recorre la misma cadena, sin saltarse pasos:

1. Problema.
2. Causa probable.
3. Evidencia.
4. Comportamiento observable.
5. Clasificación (según la tabla anterior).
6. Solución.
7. Impacto sobre el gameplay.
8. Prueba.

> **Principio.** «Se siente lento» no es una causa. Hay que medir timing, carga, objetos, colisiones y tareas del motor antes de afirmar por qué algo ocurre.

3.3 Procedimiento de observación

El estudio del juego sigue cinco pasos. Documentan comportamiento, no código.

1. **Identificación del cartucho.** Leer el encabezado iNES (16 bytes) para obtener mapper, tamaño de PRG/CHR y mirroring. Herramientas: editores hexadecimales y bases de datos de cabeceras.
2. **Observación en emulador con depuración.** Usar un emulador con visor de memoria, breakpoints y trazas para registrar qué direcciones cambian en cada evento del juego (pausa, disparo, colisión).

3. **Mapeo de estados.** Documentar por escrito la máquina de estados observada (título, juego, pausa, game over) con transiciones y disparadores. Esto es comportamiento, no código.
4. **Medición de tiempos.** Contar frames entre eventos para estimar velocidades, cadencias de disparo y tiempos de invulnerabilidad. Registrar los resultados en tablas propias.
5. **Verificación independiente.** Cada hipótesis debe comprobarse con al menos dos observaciones reproducibles antes de incluirla en la documentación como hecho.

3.4 Plantilla de registro

Ejemplo ficticio del formato con el que se registra cada hallazgo (la plantilla completa está en el Apéndice D):

ID	Evento observado	Frames medidos	Hipótesis	Verificado
E-01	Cadencia de disparo	8	Temporizador de 8 frames	Sí (3 pruebas)
E-02	Pausa	—	Bandera en RAM de estado	Pendiente

3.5 Limitación, bug o decisión de diseño

Antes de tocar cualquier comportamiento hay que separar tres conceptos que a menudo se confunden:

Concepto	Definición	Ejemplo
Limitación	Algo que el hardware o la arquitectura impone.	Cantidad máxima de entidades simultáneas (hipotético).
Bug	Una implementación que produce un comportamiento incorrecto.	Glitch del Laser Wall.
Decisión de diseño	Algo que parece una limitación pero puede haber sido deliberado.	Limitar la cantidad de proyectiles para controlar la dificultad.

> **Regla.** No se corrige nada hasta saber a cuál de las tres categorías corresponde. Una limitación puede desaparecer en PC; un bug puede corregirse en el modo Enhanced; una decisión de diseño se conserva.

PARTE

II

Hardware y cartucho

El hardware de la NES y la identificación de la ROM.

Capítulos de esta parte

- 4. Arquitectura de hardware NES
- 5. Identificación del juego y de la ROM

CAPÍTULO

04

Arquitectura de hardware NES

El hardware de la NES que condiciona el juego: CPU, PPU, APU, mapa de memoria y por qué importan para la reimplementación.

En este capítulo

- 4.1 Componentes
- 4.2 Mapa de memoria de la CPU (resumen)
- 4.3 Por qué importa para el proyecto

Para reimplementar B-Wings hay que distinguir qué comportamientos nacen del diseño del juego y cuáles del hardware que lo ejecutaba. Este capítulo resume el hardware relevante; las referencias públicas están en el Apéndice B.

4.1 Componentes

Componente	Función	Detalle relevante para reimplementar
CPU Ricoh 2A03	Núcleo 6502 sin modo decimal	~1,79 MHz (NTSC); 2 KB de RAM interna
PPU 2C02	Gráficos	256×240 px, 64 colores de paleta, 8 sprites por línea
APU	Sonido	2 pulsos, triángulo, ruido, canal DMC
Cartucho / mapper	Bancos de PRG y CHR	Determina cuánta ROM y VRAM se pueden direccionar
Controladores	Entrada	Registros `$4016/$4017`, lectura serial de 8 botones

4.2 Mapa de memoria de la CPU (resumen)

Rango	Contenido
`$0000–$07FF`	RAM interna (espejada hasta `$1FFF`)
`$2000–$2007`	Registros de la PPU (espejados hasta `$3FFF`)
`$4000–$4017`	APU, DMA de sprites (`$4014`) y controladores
`$4020–$FFFF`	Espacio de cartucho (PRG-RAM, PRG-ROM, vectores)

4.3 Por qué importa para el proyecto

Con unos 2 KB de RAM y un presupuesto de CPU fijo por cuadro, cualquier juego de NES reparte su trabajo en tareas que deben terminar antes del siguiente cuadro; cuando no terminan aparecen el slowdown y la pérdida de sincronía (capítulo 17). La PPU, además, sólo puede mostrar 8 sprites por línea, lo que explica el parpadeo típico de la plataforma. Estas son **limitaciones de hardware** en el sentido de la sección 3.5: en PC pueden desaparecer, pero sólo después de comprobar que no forman parte del diseño del juego.

CAPÍTULO

05

Identificación del juego y de la ROM

La identidad exacta de la ROM estudiada y el mapper 185 con su protección de CHR-ROM.

En este capítulo

- 5.1 El mapper 185

Fuente. B-Wings fue desarrollado por Data East. La versión Famicom de 1986 contiene 30 niveles y adapta el arcade de 1984 [1].

ESPACIO RESERVADO PARA CAPTURA

Ilustra: Identificación del juego (sección 5)

Foto 1. *Pantalla de título y cartucho Famicom de B-Wings*

Dato	Valor
Archivo	B-Wings (J).nes
PRG-ROM	32 KiB
CHR-ROM	8 KiB
PRG CRC32	725D5CC4
CHR CRC32	E4075FC0
Mapper	185
Mirroring	Horizontal

5.1 El mapper 185

Fuente. El mapper 185 utiliza lógica de protección que puede deshabilitar selectivamente la CHR-ROM; por eso algunas escrituras del arranque son comunicación con el cartucho y no variables normales del juego [3].

Consecuencia. La protección se estudia para entender la ROM, pero no se copia como gameplay en PC.

PARTE

III

El juego

Las reglas del juego: armas, Morph, estructuras, enemigos, etapas y bosses.

Capítulos de esta parte

- 6. Modelo de gameplay
- 7. Sistema de WINGs
- 8. Sistema Morph
- 9. Estructuras
- 10. Enemigos
- 11. Etapas
- 12. Bosses

CAPÍTULO

06

Modelo de gameplay

Cómo se juega B-Wings: las dos alturas, el scroll cíclico, la selección inicial de WING y sus estados.

En este capítulo

- 6.1 Dos alturas de juego
- 6.2 Scroll lateral cíclico
- 6.3 Selección inicial de WING
- 6.4 Estados de WING

El jugador controla el FX-1 en un shooter vertical y obtiene módulos WING que cambian radicalmente su capacidad ofensiva. La versión Famicom tiene 30 etapas [1][F-A].

6.1 Dos alturas de juego

La documentación describe dos alturas: sin WING se puede descender a un nivel inferior, donde ciertos enemigos aéreos dejan de afectar al jugador, aunque siguen existiendo amenazas terrestres y obstáculos [F-A].

ESPACIO RESERVADO PARA CAPTURA

Ilustra: Sección 6.1

Foto 2. *Las dos alturas de juego: con WING y sin WING*

6.2 Scroll lateral cíclico

La pantalla continúa lateralmente de forma cíclica; por tanto, el movimiento horizontal no debe modelarse como un simple rectángulo con dos paredes [F-A].

6.3 Selección inicial de WING

La selección inicial de WING aparece en las etapas 1, 5, 9, 13, 17, 21, 25 y 29. El patrón 4n+1 está documentado [F-B].

Etapa con selección	1	5	9	13	17	21	25	29
n (en 4n+1)	0	1	2	3	4	5	6	7

ESPACIO RESERVADO PARA CAPTURA

Ilustra: Sección 6.3 (etapas 4n+1)

Foto 3. *Pantalla de selección de WING*

6.4 Estados de WING

La pérdida de un WING no equivale necesariamente a la pérdida inmediata de una vida. El sistema posee estados diferenciados de WING equipado, WING perdido, destrucción y recuperación. Las transiciones exactas deben verificarse directamente antes de fijarlas (**PENDIENTE**).

CAPÍTULO

07

Sistema de WINGs

El sistema de armas completo: catálogo, selector $00CF, armas ocultas, códigos publicados y parámetros por WING.

En este capítulo

- 7.1 Catálogo de WINGs
- 7.2 El selector `$00CF`
- 7.3 Una capa de armas especiales sobre el selector común
- 7.4 Códigos publicados y qué revelan
- 7.5 Parámetros por WING que deben reconstruirse
- 7.6 Los WINGs como datos

Los WINGs son sistemas de comportamiento, no simples cambios de sprite. Alteran cantidad, dirección, geometría, penetración, defensa y reglas de colisión. Por eso el manual los trata como un sistema con capas y no como una lista de nombres de armas.

7.1 Catálogo de WINGs

WING	Función / comportamiento a verificar
FX-1	Disparo básico; límite y cadencia deben medirse.
Cannon	Tres proyectiles en rápida sucesión.
Wide	Disparo abierto; regla especial al impactar.
Multi	Proyectiles con diferentes ángulos.
Van	Barrera de proyectiles de alcance corto.
Side	Ataques laterales y frontal.
Anti	Ataques hacia delante y atrás.
Hammer	Disparo + elementos orbitales defensivos.
Jump	Proyectiles que saltan obstáculos.
Dyna	Rayo corto con capacidad de destrucción continua.
Fire	Llamas capaces de penetrar enemigos y bosses.
Ocultas	Ring Laser, Multi-Hammer y X-Ray (según el manual v0.2); comportamiento exacto pendiente de medición. Ver la reconciliación con los nombres del selector en la sección 7.3 y en el Apéndice C.

> **Fuente.** Las funciones generales de estas WINGs están corroboradas por documentación de gameplay [2][4].

ESPACIO RESERVADO PARA CAPTURA

Ilustra: Sección 7.1

Foto 4. *Catálogo de WINGs en acción (una captura por WING)*

7.2 El selector `$00CF`

La evidencia independiente confirma que `$00CF` funciona como selector de WING. Está documentado directamente por Whipon y aparece también en bases independientes de códigos. Clasificación: **HECHO**, confianza Alta.

$00CF	WING
$01	Cannon
$02	Wide
$03	Multi
$04	Van
$05	Side
$06	Anti
$07	Hammer
$08	Jump
$09	Dyna
$0A	Fire
$0B	Hammer Laser
$0C	4 Way Laser Wall
$0D	Multi Hammer

Las fuentes consultadas no documentan el valor `$00`. No se asume su significado (por ejemplo, «sin WING» o FX-1 básico): queda **PENDIENTE**.

7.3 Una capa de armas especiales sobre el selector común

Lo realmente interesante es lo que aparece alrededor del selector. Los códigos publicados documentan:

- Start With Hidden Weapon 1 y Start With Hidden Weapon 2.
- Enable Secret Weapons Morphs.
- Enable Unlimited Morphs.
- Can Morph Always.
- Don't Lose Your Weapon After Crashing.

Una guía de códigos especifica qué hacen las dos armas ocultas:

Arma oculta	Comportamiento descrito	Valor asociado
Hidden Weapon 1	Produce grandes burbujas y tiene dos bolas que protegen al jugador como escudo.	...01/0B → `$00CF` = `$0B`
Hidden Weapon 2	Dispara ocho bolas en diferentes direcciones.	...01/0D → `$00CF` = `$0D`

Otra fuente que traduce los códigos internos del emulador muestra que los códigos de inicio para las armas ocultas modifican valores terminados en 0B y 0D, lo que coincide exactamente con `$00CF` = `$0B` y `$00CF` = `$0D`. Esto permite establecer con bastante confianza que **0B y 0D no son valores arbitrarios de un cheat, sino estados funcionales del sistema de armas**.

> **Precisión importante.** No debe llamarse a 0B, 0C y 0D simplemente «WINGs normales». El juego parece tener una capa de armas especiales o secretas encima del selector común. La relación exacta entre estos tres valores y los nombres Ring Laser, Multi-Hammer y X-Ray del manual v0.2 está pendiente (Apéndice C).

ESPACIO RESERVADO PARA CAPTURA

Ilustra: Sección 7.3

Foto 5. *Armas ocultas: burbujas con bolas protectoras y disparo de ocho bolas*



Figura 1. *Arquitectura probable del sistema de armas (hipótesis de investigación)*

El diagrama resume la arquitectura probable del sistema de armas: el selector `$00CF` reparte el control entre WINGs normales (01–0A), armas especiales (0B y 0D) y el Laser Wall (0C), y las tres familias terminan en el sistema de objetos y proyectiles. Es un modelo de investigación, mucho más fuerte que el anterior, pero todavía no una descripción definitiva del código.

7.4 Códigos publicados y qué revelan

Código publicado	Dirección / valor	Qué sugiere	Clasificación
Start With Hidden Weapon 1 / 2	Valores ...01/0B y ...01/0D	`$00CF` = `$0B` y `$0D` son armas reales del juego	HECHO
Enable Secret Weapons Morphs	No registrada	Las armas secretas pueden usarse durante Morph	PENDIENTE

Código publicado	Dirección / valor	Qué sugiere	Clasificación
Enable Unlimited Morphs / Can Morph Always	No registrada	La cantidad de Morph y la posibilidad de hacerlo son conceptos distintos	INFERENCIA
Infinite Morphs	`$0541` = `$09`	Posible recurso o estado de Morph (ver capítulo 8)	INFERENCIA
Don't Lose Your Weapon After Crashing	No registrada	Por defecto el arma se pierde al chocar: existe un estado de «crash»	INFERENCIA
Increase Van Range / Increase Dyna Range	No registrada	El alcance de las armas es un parámetro modificable	INFERENCIA
Disable 4 Way Laser Wall Glitch	`$00A0` = `$FF`, `$00A1` = `$FF`	Estado asociado al glitch (ver sección 18.1)	HECHO (relación) / PENDIENTE (causa)

7.5 Parámetros por WING que deben reconstruirse

Los códigos Increase Van Range e Increase Dyna Range muestran que el comportamiento de las armas no está necesariamente codificado como una propiedad indivisible. Probablemente existan parámetros o tablas para alcance, velocidad, cantidad, dirección, duración, tipo de proyectil y patrón de disparo (HIPÓTESIS).

Esto abre una línea de investigación concreta: para cada WING hay que reconstruir una ficha completa, mucho más útil para la implementación que una lista de nombres.

Campo de la ficha de WING	Qué debe registrar
Código / ID	Valor de `$00CF` y nombre
Proyectil	Tipo y apariencia lógica del objeto creado
Cantidad simultánea	Proyectiles o elementos activos a la vez
Velocidad	Desplazamiento por tick
Dirección	Ángulos o patrón de salida
Alcance	Distancia o duración máxima de vuelo
Duración	Tiempo de vida de los objetos persistentes
Cadencia	Frames entre disparos
Comportamiento al impactar	Qué ocurre al golpear enemigos
Interacción con estructuras	Puntos débiles, sectores y reglas especiales
Interacción con bosses	Daño, penetración y fases
Después de Morph / crash	Estado del arma tras transformarse o chocar

7.6 Los WINGs como datos

> **Arquitectura.** Cada WING debe representarse como datos: patrón de disparo + proyectiles + colisión + defensa + reglas especiales + visual. El motor interpreta esos datos; no hay una clase de código por arma.

CAPÍTULO

08

Sistema Morph

El sistema Morph y por qué $0541 todavía no puede llamarse contador.

En este capítulo

- 8.1 Tres conceptos que no deben mezclarse
- 8.2 Qué puede ser `$0541`

`$0541` probablemente pertenece al sistema Morph, pero todavía no debe llamarse «contador». Hay un código publicado que lo fija en `$09` (Infinite Morphs), y además existen códigos separados: Enable Unlimited Morphs, Enable Secret Weapons Morphs y Can Morph Always. Eso indica que hay al menos tres conceptos distintos detrás de la palabra «Morph».

8.1 Tres conceptos que no deben mezclarse

Concepto	Pregunta que debe responder la investigación	Código publicado relacionado
Cantidad / recurso	¿Cuántos Morph quedan y cómo se gastan?	Infinite Morphs (`$0541` = `$09`), Enable Unlimited Morphs
Posibilidad de hacer Morph	¿Qué condiciones habilitan la transformación?	Can Morph Always
Armas permitidas durante Morph	¿Qué valores de `$00CF` pueden usarse transformado?	Enable Secret Weapons Morphs

> **Advertencia para la implementación.** Sería un error escribir `int morphs = 9;` en el PC y dar por cerrado el análisis. Primero hay que descubrir qué hace realmente `$0541`.

8.2 Qué puede ser `$0541`

Candidatos a verificar: contador, estado, máscara, cantidad de Morph, disponibilidad o una combinación de funciones según el contexto. El manual v0.2 lo clasificaba como «estado morph/WING» con confianza Alta; esta edición lo baja a **Medio/Alto** como parte del Morph y **sin función definida**, hasta resolver los accesos descritos en la sección 19.2.

CAPÍTULO

09

Estructuras

Estructuras compuestas, puntos débiles y colisión basada en reglas.

Las estructuras no son simplemente enemigos grandes. Muchas tienen sectores, puntos débiles, direcciones de ataque o interacciones específicas.

Familias documentadas	Tratamiento recomendado
H, Octagon, Tap, Cable	Entidades compuestas con partes y puntos vulnerables.
Side, Neon, Sputnik	Múltiples segmentos o vulnerabilidades direccionales.
Land Mine	Colisión amplia.
Solid	Regla especial asociada a Jump.
Delta	Barricada con punto protegido.
Mix & Match	Composición de estructuras.



Foto 6. *Estructuras: H, Octagon, Delta y Mix & Match*

> **Conclusión.** La colisión debe ser un sistema de reglas, no una sola caja por sprite.

CAPÍTULO

10

Enemigos

Las familias de enemigos y los datos que hay que medir para modelarlos como máquinas de estados.

En este capítulo

- 10.1 Datos a medir por enemigo

La documentación histórica enumera numerosas familias: Bombs, Bats, Sirens, Octopi, Diamonds, Syringes, Cabbages, Barrels, Homing Drills, Darts, Spiked Cymbals, Cupcakes, Toy Aeroplanes, Frying Pans, Retractors, Stern Bats, Space Shuttles, Caterpillars, Carrots y Y-Flyers.

Cada enemigo deberá modelarse mediante estados: spawn, entrada, movimiento, ataque, vulnerabilidad, retirada, muerte y eliminación.



Foto 7. *Familias de enemigos (Bombs, Bats, Sirens y otras)*

10.1 Datos a medir por enemigo

- Posición de aparición.
- Momento de activación.
- Trayectoria.
- Velocidad y aceleración.
- Seguimiento al jugador.
- Cadencia y cantidad de disparos.
- Hitbox.
- Duración de estados.
- Condición de retirada.
- Puntuación.

CAPÍTULO

11

Etapas

Las 30 etapas convertidas en datos: fondo, scroll, estructuras, enemigos, WING y boss.

Las 30 etapas deben convertirse en datos: fondo, perfil de scroll, secuencia de estructuras, enemigos, objetos, WING y boss. El motor interpreta esos datos; no hay lógica de nivel escrita a mano.

> **Fuente.** La existencia de 30 etapas está corroborada por fuentes independientes; también existe un playthrough completo útil para comparación visual [1][5]. Las observaciones históricas sobre mayor cantidad de enemigos y proyectiles en etapas avanzadas son una pista para estudiar el slowdown, no una prueba de causalidad (capítulo 17).

ESPACIO RESERVADO PARA CAPTURA

Ilustra: Capítulo 11

Foto 8. *Fondos y primeras etapas del juego*

Campo de datos por etapa	Contenido
Fondo	Gráficos de fondo y perfil visual
Perfil de scroll	Velocidad y formato de desplazamiento
Secuencia de estructuras	Orden y posición
Enemigos y objetos	Tipos, tiempos de aparición, trayectorias
WING	Selección o entrega de WING (etapas 4n+1 para la selección)
Boss	Tipo, variante de etapa y reglas

CAPÍTULO

12

Bosses

Los Gobunasu y el modelo de boss por tipo, variante y fases.

Los Gobunasu aparecen al final de las etapas. No se debe asumir que cada jefe tiene una vida fija independiente de la etapa. El modelo previsto es:

> **Modelo de boss.** tipo de boss + variante de etapa + reglas de impacto + vulnerabilidad + fases + proyectiles + condición de derrota.



Foto 9. *Jefe de fin de etapa (Gobunasu)*

PENDIENTE. Registrar golpes, WING utilizado, puntos vulnerables, fases y proyectiles de cada jefe.

PARTE

IV

Ingeniería inversa de la ROM

Lo que muestra la ROM: arquitectura interna, memoria, proyectiles, slowdown y glitches.

Capítulos de esta parte

- 13. Arquitectura interna observada
- 14. Mapa de RAM
- 15. Scrolling
- 16. Proyectiles y entidades
- 17. Slowdown
- 18. Glitches y colisiones anómalas
- 19. Agenda de investigación en la ROM

CAPÍTULO

13

Arquitectura interna observada

La NMI, el dispatcher de 25 tareas, el manejo de sprites y la lectura del mando.

En este capítulo

- 13.1 Tareas del dispatcher
- 13.2 Sprites y OAM
- 13.3 Entrada y movimiento del jugador

El análisis previo identificó un sistema de NMI, contadores de sincronización y un dispatcher de 25 tareas. Conceptualmente el ciclo es: NMI → avance temporal → tareas del motor → marcador de iteración → siguiente sincronización. Confianza en esta arquitectura: **Alta**.



Figura 2. *Ciclo NMI y dispatcher: 25 tareas, 4 identificadas*

13.1 Tareas del dispatcher

Índice	Dirección	Identificación actual
00	B3BC	Pendiente
01	86DE	Pendiente
02	877A	Pendiente
03	871B	Pendiente
04	889F	Construcción de OAM/sprites — alta
05	8857	Movimiento de alas/objetos — alta
06	84EA	Movimiento del jugador — alta
07	853B	Representación del jugador — alta
08	A971	Pendiente
09	9F18	Pendiente
10	9FA5	Pendiente
11	9FB7	Pendiente
12	9E16	Pendiente
13	9095	Pendiente
14	9154	Pendiente
15	9D3F	Pendiente
16	A537	Pendiente
17	A126	Pendiente
18	A3A6	Pendiente

Índice	Dirección	Identificación actual
19	A432	Pendiente
20	A47A	Pendiente
21	AD1B	Pendiente
22	B357	Pendiente
23	8457	Pendiente
24	84B2	Pendiente

De las 25 tareas, 4 están identificadas con confianza alta y 21 siguen pendientes; completar esta tabla es la primera prioridad de la agenda de investigación (capítulo 19).

13.2 Sprites y OAM

La rutina alrededor de `$889F` construye OAM en RAM y la NMI utiliza `$4014` para transferirlo a la PPU. En PC conviene conservar la separación entre simulación y renderizado.

13.3 Entrada y movimiento del jugador

El mando se lee mediante `$4016`; `$00D1` recibe el estado. La rutina alrededor de `$84EA` utiliza `$00D1 & $0F` como índice de tablas de movimiento. Las tablas alrededor de `$C5C7/$C5D7` deben decodificarse antes de fijar la física. Estado del conocimiento del movimiento del jugador: **parcial**.

CAPÍTULO

14

Mapa de RAM

Las direcciones de RAM identificadas, con su nivel de confianza antes y después de los últimos hallazgos.

En este capítulo

- 14.1 Registros de hardware que usa el juego

El mapa reúne las direcciones identificadas hasta ahora, con la confianza del manual v0.2 y su actualización tras los hallazgos recientes sobre WINGs, Morph y Laser Wall.

Dirección	Interpretación	Confianza v0.2	Actualización (esta edición)
`$0015`	Contador NMI / temporal	Alta	Sin cambios
`$0016`	Marcador de sincronización	Alta	Sin cambios
`$0024`	Iteración del motor	Alta	Sin cambios
`$009A`	Array / slots de objetos y alas	Alta	Posible pool de entidades: **PENDIENTE** (capítulo 16)
`$00A0-$00A1`	Estado relacionado con Laser Wall / entidades especiales	Media	Modificados por el código Disable 4 Way Laser Wall Glitch (`$FF`); confianza Medio/Alto; significado exacto **PENDIENTE**
`$00CD`	X del jugador	Alta	Sin cambios
`$00CE`	Y del jugador	Alta	Sin cambios
`$00CF`	Estado / identificador de WING	Alta	Confirmado como selector de WING, valores `$01` a `$0D` (sección 7.2)
`$00D1`	Entrada	Alta	Sin cambios
`$00D6`	Participa en el scroll	(sin nivel asignado)	Ver capítulo 15
`$053E`	Vidas	Alta	Sin cambios
`$0541`	Estado Morph / WING	Alta	Publicado como Infinite Morphs (`$09`); función por definir; confianza Medio/Alto (capítulo 8)
`$0543-$0547`	Scroll / progresión	Media	Sin cambios

14.1 Registros de hardware que usa el juego

Registro	Uso observado
`$2000` PPUCTRL, `$2001` PPUMASK, `$2005` PPUSCROLL	Escritos por la NMI para el scroll y la visualización
`$4014`	DMA de OAM: transferencia de los sprites construidos en RAM
`$4016`	Lectura del mando; el resultado queda en `$00D1`

CAPÍTULO

15

Scrolling

Lo que se sabe del scroll y lo que debe reconstruirse antes de implementarlo.

La NMI escribe PPUCTRL (`$2000`), PPUMASK (`$2001`) y PPUSCROLL (`$2005`). `$00D6` participa en el scroll y `$0543-$0547` participan en la progresión y en cálculos gráficos. Una rutina alrededor de `$AD00` combina `$0543/$0544` mediante desplazamientos y máscaras. Estado del conocimiento: **parcial**.

> **Pendiente crítico.** Todavía no debe suponerse una fórmula de scroll de píxeles enteros por frame. Hay que reconstruir el formato exacto antes de implementar el desplazamiento. Recordar además que el movimiento horizontal es cíclico (sección 6.2).

CAPÍTULO

16

Proyectiles y entidades

Si los proyectiles son balas o entidades: la pregunta que condiciona límites, slots, colisiones y slowdown.

Los hallazgos sobre armas obligan a separar algo que hasta ahora estaba demasiado junto: ¿los proyectiles son «balas» simples o son entidades? El Hidden Weapon 2 dispara ocho bolas en direcciones distintas, y el Hidden Weapon 1 mantiene dos bolas que funcionan como protección alrededor del jugador. Si los proyectiles son entidades independientes, el modelo sería:

```
PLAYER
  └── FIRE
        ├── projectile slot
        ├── projectile slot
        ├── projectile slot
        └── ...
```

El Hammer podría incluso conservar objetos activos después de un disparo. Este punto es especialmente importante porque condiciona todo lo siguiente:

Tema	Pregunta abierta
Límite de proyectiles	¿Existe un pool fijo de slots? ¿De cuántos?
Creación y destrucción	¿Cuándo nace y cuándo se elimina cada objeto?
Reutilización de slots	¿Qué ocurre cuando el pool está lleno?
Prioridad	¿Qué objetos desplazan a cuáles?
Colisiones	¿Cómo y cuándo se evalúan para cada tipo de proyectil?
Slowdown	¿Qué coste añade cada tipo de entidad?
Bugs	¿Qué glitches nacen de compartir slots entre tipos de objeto?

Estado actual: sistema de proyectiles **PENDIENTE**; pool de entidades **PENDIENTE** (la dirección `$009A` es un candidato a array de slots de objetos y alas). Si existe un pool fijo, será una pieza fundamental para explicar las limitaciones originales del juego y para decidir qué límites pertenecen al diseño y cuáles al hardware (sección 3.5).

CAPÍTULO

17

Slowdown

La causa del slowdown como hipótesis a demostrar, el coste de cada WING y un experimento para medirlo.

En este capítulo

- 17.1 El coste no es el mismo para todos los WINGs
- 17.2 Experimento propuesto

El slowdown es un comportamiento perceptible corroborado por referencias históricas, pero su causa exacta todavía no está demostrada.

ESPACIO RESERVADO PARA CAPTURA

Ilustra: Capítulo 17

Foto 10. *Escena con carga alta de enemigos y proyectiles (slowdown)*

Afirmación	Estado
Existe slowdown/lag perceptible	Hecho corroborado externamente
Etapas avanzadas concentran más elementos	Observación
Los objetos causan el slowdown	Hipótesis
Las colisiones son la causa principal	Hipótesis
NMI/dispatcher intervienen	Hipótesis de investigación
Enhanced debe eliminar una limitación técnica comprobada	Decisión del proyecto

17.1 El coste no es el mismo para todos los WINGs

Si cada WING crea objetos de distinto tipo, entonces no todos tienen el mismo coste computacional:

WING	Tipo de carga que probablemente genera
Cannon	Pocos proyectiles
Multi	Varios proyectiles
Van	Múltiples direcciones
Hammer	Objetos persistentes
Multi Hammer	Múltiples objetos persistentes
Laser Wall	Objetos y efectos especiales

Por eso, al estudiar el slowdown no alcanza con contar enemigos. Hay que medir la suma:

> **Carga total.** enemigos + proyectiles enemigos + proyectiles del jugador + objetos persistentes + colisiones + efectos especiales

Esto podría explicar por qué determinadas combinaciones de enemigos y armas producen más carga, pero sigue siendo **HIPÓTESIS**, no causa demostrada.

17.2 Experimento propuesto

1. Registrar NMI/frame por tick.
2. Contar enemigos, proyectiles y estructuras.
3. Medir tiempo por familia de rutina.
4. Detectar tareas incompletas.
5. Repetir con carga creciente.
6. Comparar el estado interno con el ritmo visual.

> **Objetivo.** Encontrar la causa, no simplemente aumentar la velocidad.

CAPÍTULO

18

Glitches y colisiones anómalas

El glitch del 4 Way Laser Wall y las colisiones que parecen alcanzar al jugador sin contacto.

En este capítulo

- 18.1 Laser Wall
- 18.2 Colisiones cercanas

18.1 Laser Wall

Existe documentación del llamado 4 Way Laser Wall Glitch. `$00CF` = `$0C` selecciona el 4 Way Laser Wall, y además existe un código publicado, Disable 4 Way Laser Wall Glitch, que fija `$00A0` = `$FF` y `$00A1` = `$FF`. Hay por tanto dos grupos de variables: el selector del arma (`$00CF`) y un estado relacionado con el glitch (`$00A0/$00A1`).

Clasificación	Afirmación
HECHO	`$00A0/$00A1` son modificados por un código cuyo propósito publicado es eliminar el glitch del 4 Way Laser Wall.
INFERENCIA	Participan directamente en el estado que permite que aparezca el comportamiento defectuoso.
PENDIENTE	Determinar qué representan exactamente y qué rutina los escribe y los lee.

Lo que todavía no puede afirmarse. No se sabe si `$00A0/$00A1` son el estado del arma, coordenadas, flags internos o slots de entidades generadas por el Laser Wall. La evidencia sólo demuestra una relación funcional muy fuerte. Por eso la clasificación se mantiene así, para no contaminar la reconstrucción con una suposición.

Clasificación del glitch: candidato a bug; causa no demostrada.

ESPACIO RESERVADO PARA CAPTURA

Ilustra: Sección 18.1

Foto 11. *Laser Wall en acción (4 Way Laser Wall)*

18.2 Colisiones cercanas

Fuentes históricas describen proyectiles que parecen destruir al jugador sin contacto visual. Posibles explicaciones: hitbox mayor, evaluación en momentos distintos, posición interna diferente al sprite o una condición especial.

ESPACIO RESERVADO PARA CAPTURA

Ilustra: Sección 18.2

Foto 12. *Proyectil que parece alcanzar al jugador sin contacto visual*

> **Regla.** No corregir hasta reproducir y explicar.

CAPÍTULO

19

Agenda de investigación en la ROM

Los cuatro frentes de investigación en la ROM y la lista general de pendientes.

En este capítulo

- 19.1 `$00CF`: el árbol de decisiones de las armas
- 19.2 `$0541`: qué es realmente
- 19.3 `$00A0/$00A1`: todos sus accesos
- 19.4 Sistema de proyectiles
- 19.5 Lista general de pendientes prioritarios

La prioridad actual se concentra en cuatro frentes, seguidos de la lista general de pendientes.

19.1 `$00CF`: el árbol de decisiones de las armas

Buscar todos los sitios donde se haga `LDA $00CF`, `CMP #valor`, `AND $00CF`, `STA $00CF` y similares. Objetivo: construir el árbol real de decisiones de las 13 armas.

19.2 `$0541`: qué es realmente

Buscar `LDA $0541`, `STA $0541`, `INC $0541`, `DEC $0541`, `CMP #$09` y equivalentes. Objetivo: determinar si es contador, estado, máscara, cantidad de Morph, disponibilidad o una combinación de funciones según el contexto.

19.3 `$00A0/$00A1`: todos sus accesos

Localizar absolutamente todos sus accesos: quién los escribe, cuándo, quién los lee y qué decisiones provocan. Es probablemente el mejor camino para explicar de verdad el glitch del Laser Wall.

19.4 Sistema de proyectiles

Localizar creación, actualización, colisión y destrucción de proyectiles, y determinar si existe un pool fijo de slots.

19.5 Lista general de pendientes prioritarios

1. Identificar las 25 tareas del dispatcher.
2. Decodificar movimiento y representación de posición.
3. Mapear `$00CF` y los estados de WING.
4. Localizar slots y límites de proyectiles.
5. Clasificar objetos y enemigos internos.
6. Reconstruir estructuras y puntos débiles.
7. Decodificar HP y fases de los bosses.
8. Decodificar las tablas de las 30 etapas.
9. Medir el scroll.
10. Correlacionar carga con slowdown.
11. Reproducir el Laser Wall de forma causal.
12. Reproducir las colisiones cercanas.
13. Medir todos los WINGs.
14. Construir la tabla numérica final.

PARTE

V

Reimplementación en PC

Cómo reconstruirlo en PC: modos, arquitectura, pruebas y plan.

Capítulos de esta parte

- 20. Qué conservar y qué mejorar
- 21. Arquitectura propuesta
- 22. Validación y pruebas de regresión
- 23. Plan de desarrollo
- 24. Documentación, distribución y próximos pasos

CAPÍTULO

20

Qué conservar y qué mejorar

Qué debe conservarse del original y qué puede mejorarse en el modo Enhanced.

Elemento	Original	Enhanced
Timing	Comportamiento medido	Estable
Input	Misma lógica temporal	Respuesta consistente
Scroll	Misma progresión	Sin jitter
Sprites	Composición equivalente	Sin límite artificial de OAM
Proyectiles	Límites de diseño	Eliminar sólo límites técnicos demostrados
Slowdown	Reproducir si es comportamiento confirmado	Eliminar la causa técnica
Colisiones	Reglas medidas	Correcciones sólo como opción explícita

> **Distinción.** Un límite de hardware puede desaparecer; una regla de diseño no debe cambiar sólo porque el PC sea más potente.

El modo Rebuild / Extendido (sección 1.2) se apoya en la misma base: añade niveles, enemigos y más disparos en pantalla sin alterar los perfiles Original y Enhanced.

CAPÍTULO

21

Arquitectura propuesta

La arquitectura por capas, la estructura de proyecto en Godot 4 y las recomendaciones técnicas.

En este capítulo

- 21.1 Capas
- 21.2 Estructura de proyecto sugerida (Godot 4)
- 21.3 Recomendaciones técnicas
- 21.4 Elección de tecnología

21.1 Capas

Capa	Responsabilidad
Core / Timing	Tick, reloj, sincronización, Original / Enhanced
Input	Entrada y registro temporal
Simulation	Jugador, WINGs, proyectiles, enemigos, estructuras, items, bosses
Stage	Datos de las 30 etapas
Collision	Geometría y reglas especiales
Rendering	Pixel art, fondos, sprites, cámara
Audio	Música y efectos
Telemetry	Contadores, hitboxes, comparación
Presentation	Ventana, resolución, fullscreen, configuración

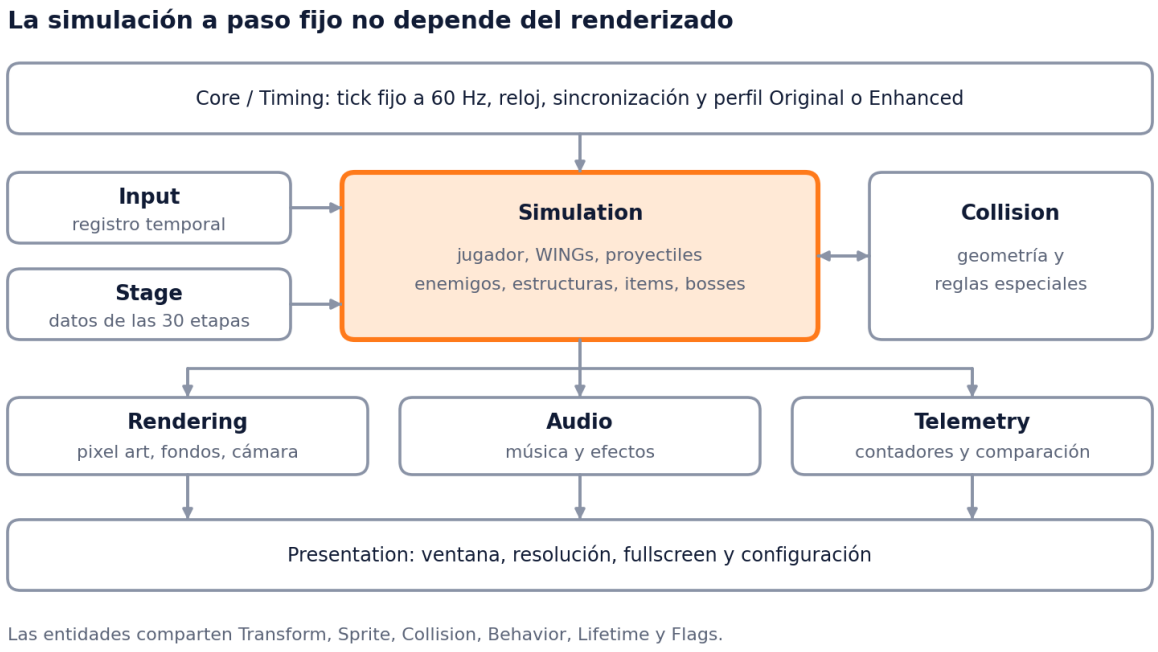


Figura 3. *Arquitectura propuesta por capas: la simulación a paso fijo no depende del renderizado*

Las entidades deberían compartir Transform, Sprite, Collision, Behavior, Lifetime y Flags. Los comportamientos específicos deben estar guiados por datos y estados, no por una clase de código por enemigo.

21.2 Estructura de proyecto sugerida (Godot 4)

```
res://
├── core/       # bucle de juego a 60 Hz fijo, estados, RNG con semilla
├── entities/   # jugador, enemigos, proyectiles (lógica pura, sin nodos)
├── collision/  # detección AABB y rejillas espaciales
├── levels/     # formato de datos propio (JSON o Resource)
├── render/     # presentación con escalado entero y filtros opcionales
├── input/      # mapeo de acciones configurable
├── audio/      # reproductor de eventos sonoros propios
└── modes/      # original, corregido, extendido (activados por configuración)
```

21.3 Recomendaciones técnicas

- **Separar lógica y presentación.** La simulación debe correr en pasos fijos de frame, independiente de la tasa de renderizado. Así se pueden ofrecer 60 FPS nativos sin alterar la física.
- **Modos configurables.** En lugar de «versiones» separadas, usar perfiles de parámetros (velocidades, cadencias, número de enemigos) cargados desde archivos de datos.
- **Formato de niveles propio.** Definir un esquema JSON con validación, para que cualquier persona pueda crear niveles sin tocar el código.
- **Pruebas de regresión.** Grabar entradas y comparar estados del juego entre versiones para detectar cambios no deseados.

21.4 Elección de tecnología

Godot, Unity, GameMaker o C#/C++ siguen formalmente abiertos en el plan de desarrollo. El diseño de referencia de este manual usa Godot 4; la decisión definitiva debe tomarse después de cerrar la especificación. Como la simulación es lógica pura separada de la presentación, la elección no afecta a la especificación de comportamiento.

CAPÍTULO

22

Validación y pruebas de regresión

Qué métricas registrar y cómo convertir cada hallazgo confirmado en una prueba de regresión.

Métrica	Qué registrar
Movimiento	X/Y por tick/frame
Aceleración	Diferencia de posición
Disparo	Cadencia y proyectiles simultáneos
Proyectiles	Velocidad, dirección, alcance y colisión
Enemigos	Spawn, trayectoria y estados
Scroll	Offset
Colisiones	Instante y consecuencia
Boss	Hits, fases y muerte
Slowdown	Carga frente a timing
Input	Ticks entre pulsación y efecto

> **Regla.** Cada comportamiento confirmado debe convertirse en una prueba de regresión. La verificación independiente (sección 3.3) es el requisito para pasar de hipótesis a hecho.

CAPÍTULO

23

Plan de desarrollo

Las diez fases del desarrollo y su estado actual.

Fase	Resultado	Estado
1 Investigación	Hardware, ROM, timing, objetos	En curso
2 Especificación	Parámetros cuantificados	Parcial
3 Motor base	Timing + jugador + scroll	No iniciado
4 Entidades	Enemigos, proyectiles, estructuras	No iniciado
5 WINGs	Armas y reglas	No iniciado
6 Etapas	30 niveles como datos	No iniciado
7 Bosses	Jefes y variantes	No iniciado
8 Validación	Comparador	No iniciado
9 Enhanced	Correcciones técnicas opcionales	No iniciado
10 Distribución	Windows + documentación	No iniciado



Figura 4. *Plan de desarrollo: 10 fases y su estado actual*

Orden. Sólo conviene iniciar la implementación (fase 3 en adelante) después de cuantificar lo que aún no está cerrado: dispatcher, movimiento, proyectiles, etapas, bosses, scroll, slowdown y glitches.

CAPÍTULO

24

Documentación, distribución y próximos pasos

Cómo se estructura la biblia del proyecto, las fuentes públicas y los próximos pasos.

En este capítulo

- 24.1 Estructura recomendada de la «biblia» del proyecto
- 24.2 Fuentes públicas útiles para el estudio del hardware
- 24.3 Próximos pasos
- 24.4 Cómo se incorpora un hallazgo nuevo

24.1 Estructura recomendada de la «biblia» del proyecto

1. Objetivos y alcance legal.
2. Descripción del hardware y referencias técnicas públicas.
3. Metodología de observación y registro de hallazgos propios.
4. Especificación de la reimplementación (requisitos, arquitectura, formatos de datos).
5. Modos de juego y configuración.
6. Plan de pruebas y criterios de aceptación.
7. Licencia del proyecto y créditos de assets.

Este manual cubre los puntos 1 a 6 y deja el punto 7 como tarea del plan de distribución.

24.2 Fuentes públicas útiles para el estudio del hardware

Documentación de la comunidad NESdev (wiki.nesdev.org), especificaciones del 6502 y de la PPU/APU, y herramientas de emulación de código abierto para depuración.

24.3 Próximos pasos

1. Definir la licencia del proyecto (MIT o similar) y la política de assets.
2. Crear el prototipo de bucle a 60 Hz y un sprite de prueba propio.
3. Diseñar el esquema JSON de niveles y el primer nivel de ejemplo original.
4. Escribir las especificaciones de cada sistema en secciones separadas, con pruebas.

24.4 Cómo se incorpora un hallazgo nuevo

Cada descubrimiento se registra con la plantilla del Apéndice D, se clasifica según la sección 3.1, se vuelca en el capítulo que corresponde, se actualiza el mapa de RAM (capítulo 14) y el mapa de conocimiento (capítulo 25), y se anota en el registro de cambios del Apéndice C. Las contradicciones con versiones anteriores se conservan visibles hasta resolverse.

PARTE

VI

Estado y cierre

El mapa de conocimiento y las conclusiones.

Capítulos de esta parte

- 25. Mapa de conocimiento
- 26. Conclusión

CAPÍTULO

25

Mapa de conocimiento

El estado del conocimiento por área: antes y después de los hallazgos recientes.

En este capítulo

- 25.1 Mapa del manual v0.2
- 25.2 Estado tras los hallazgos recientes

25.1 Mapa del manual v0.2

Área	Nivel	Pendiente principal
Hardware / ROM	Alto	Nada esencial
Timing / NMI	Medio-alto	Medición cuantitativa
Movimiento	Medio	Tablas exactas
WINGs	Medio	Parámetros
Proyectiles	Bajo-medio	Slots / límites
Enemigos	Medio	Mapeo interno
Estructuras	Medio	Geometría / rutinas
Scroll	Medio	Formato
Stages	Bajo-medio	Tablas
Bosses	Medio	HP / fases
Glitches	Bajo	Causalidad
Slowdown	Bajo-medio	Causa medida

25.2 Estado tras los hallazgos recientes

Área	Estado
Identificación de la ROM	HECHO
Mapper 185	HECHO
Arquitectura NMI / dispatcher	ALTO
Movimiento del jugador	PARCIAL
Scroll	PARCIAL
`$00CF` selector de WING	ALTO
WING 01–0D	ALTO
Hidden Weapon 1 / 2	ALTO
`$0541` Morph	MEDIO/ALTO
`$00A0/$00A1` Laser Wall	MEDIO/ALTO
Causa del glitch del Laser Wall	PENDIENTE
Sistema de proyectiles	PENDIENTE
Pool de entidades	PENDIENTE
Slowdown causal	PENDIENTE
Colisiones exactas	PENDIENTE
Stage data	PENDIENTE

CAPÍTULO

26

Conclusión

Las conclusiones de la investigación y la regla de oro del proyecto.

La investigación establece que B-Wings debe reconstruirse como un sistema temporal y de estados, no como una colección de sprites. Hay evidencia suficiente para definir la filosofía, la arquitectura conceptual, las categorías de objetos y el método de validación. Los hallazgos más recientes añaden que el sistema de armas tiene más capas de las que parecía: un selector común (`$00CF`), una capa de armas especiales y un caso, el Laser Wall, que combina selección y estado con un glitch documentado.

La próxima fase crítica es cuantificar aquello que aún no está cerrado: dispatcher, movimiento, proyectiles, etapas, bosses, scroll, slowdown y glitches. Sólo después conviene iniciar la implementación.

> **Regla de oro.** No programar lo que creemos que B-Wings hace. Programar lo que podamos demostrar que B-Wings hace. Si queremos corregir algo, demostrar primero el problema, explicar su causa y separar la corrección en Enhanced.

APÉNDICES

Apéndices

Glosario, fuentes, reconciliación y plantillas.

Contenido

- Apéndice A · Glosario
- Apéndice B · Fuentes y referencias
- Apéndice C · Reconciliación y registro de cambios
- Apéndice D · Plantillas de registro

APÉNDICE

A

Glosario

Los términos técnicos que usa el manual.

Término	Definición
6502	CPU del Famicom/NES
NMI	Interrupción asociada al ciclo de vídeo
PPU	Procesador de vídeo
OAM	Datos de sprites para la PPU
DMA	Transferencia a OAM mediante `$4014`
CHR-ROM	Memoria gráfica
PRG-ROM	Memoria de programa
Mapper	Lógica adicional del cartucho
Tick	Unidad lógica del motor
Frame	Actualización visual
Hitbox	Región de colisión
Dispatcher	Mecanismo que ejecuta tareas
Clean-room	Reconstrucción independiente basada en comportamiento
iNES	Formato de archivo de ROM de NES con encabezado de 16 bytes (mapper, tamaños, mirroring)
WING	Módulo que cambia el sistema ofensivo del FX-1; seleccionado por `$00CF`
Selector de WING	Variable `$00CF` con valores `$01` a `$0D`
Morph	Sistema de transformación del jugador; función exacta de `$0541` pendiente
Slot / pool	Posición reservada en un arreglo fijo de objetos; existencia del pool pendiente de demostrar
AABB	Caja de colisión alineada con los ejes
Perfil	Conjunto de parámetros (velocidades, cadencias, límites) que define un modo de juego

APÉNDICE

B

Fuentes y referencias

Las fuentes que respaldan el contenido y las que aún faltan identificar.

Ref.	Fuente	Aporta
[1]	Wikipedia — B-Wings	Contexto histórico, 30 niveles y sistema FX-1/WING
[2]	Data East Wiki	Descripción funcional de WINGs
[3]	NESdev	Mapper 185 y protección de CHR-ROM
[4]	Arcade-History	WINGs, objetos secretos y estructura de la versión Famicom
[5]	Playthrough completo de las 30 etapas	Referencia visual de comparación
[F-A]	Fuente sobre alturas de juego y scroll lateral	Referenciada en el manual v0.2 sin identificar en la lista de fuentes; pendiente de completar
[F-B]	Fuente sobre el patrón de selección 4n+1	Referenciada en el manual v0.2 sin identificar en la lista de fuentes; pendiente de completar
—	Publicaciones de códigos de Whipon y bases independientes de códigos	Selector `$00CF` y códigos de armas ocultas, Morph, Van/Dyna y Laser Wall
—	Guía de códigos de armas ocultas	Descripción de Hidden Weapon 1 y 2
—	NESdev wiki (wiki.nesdev.org), especificaciones del 6502, PPU y APU	Hardware (capítulo 4)

APÉNDICE

C

Reconciliación y registro de cambios

Las discrepancias abiertas entre los documentos de origen y el registro de cambios.

En este capítulo

- C.1 Discrepancias abiertas
- C.2 Cambios de la versión 0.3 respecto de la 0.2

Esta edición unifica tres entradas que no siempre coinciden. Las diferencias se dejan visibles en lugar de resolverlas por suposición.

C.1 Discrepancias abiertas

Tema	Manual v0.2	Hallazgos recientes	Tratamiento en esta edición
Armas ocultas	Ring Laser, Multi-Hammer y X-Ray	`$00CF` = `$0B` Hammer Laser, `$0C` 4 Way Laser Wall, `$0D` Multi Hammer	Se conservan ambas listas; la equivalencia entre nombres queda **PENDIENTE**
`$0541`	Estado morph/WING, confianza Alta	Posible parte del Morph; no confirmado como contador; Medio/Alto	Se usa el nivel más prudente
`$00A0-$00A1`	Confianza Media	Modificados por el código del glitch del Laser Wall; Medio/Alto	Se usa Medio/Alto, causa **PENDIENTE**
Modos de juego	Original y Enhanced	Guía: original, corregido y extendido	Tres modos (sección 1.2), confirmado por el proyecto
Motor	Godot, Unity, GameMaker o C#/C++ abiertos	Guía y objetivo del proyecto: Godot 4	Godot 4 como diseño de referencia (sección 21.4)
Valor `$00` de `$00CF`	No tratado	No documentado en las fuentes	**PENDIENTE**
Herramienta de los códigos internos	—	El texto de origen menciona SNES9x, un emulador de SNES	Verificar qué emulador de NES se usó realmente

C.2 Cambios de la versión 0.3 respecto de la 0.2

1. Se incorporó la guía técnica de arquitectura NES y reimplementación limpia (capítulos 2, 3, 4 y 21 a 24).
2. Se incorporó el análisis del selector `$00CF`, las armas ocultas, los códigos publicados, Morph, Laser Wall, proyectiles y slowdown (capítulos 7, 8, 16 a 19).
3. Se añadieron diagramas (sistema de armas, dispatcher, arquitectura y plan de desarrollo), el índice, el mapa de RAM ampliado y las plantillas del Apéndice D.
4. Se eliminaron los marcadores de citas automáticos del manual v0.2 y se reemplazaron por referencias numeradas (Apéndice B).
5. No se eliminó ningún contenido técnico; sólo se fusionaron los pasajes repetidos entre los documentos (por ejemplo, la estrategia clean-room y la arquitectura de capas).
6. Edición en formato libro: carátula, portadas de capítulo, recuadros de color, marcos para fotografías y diagramas integrados.

APÉNDICE

D

Plantillas de registro

Plantillas para registrar hallazgos, direcciones, WINGs y enemigos.

En este capítulo

- D.1 Registro de hallazgo
- D.2 Ficha de dirección de RAM
- D.3 Ficha de WING
- D.4 Ficha de enemigo

D.1 Registro de hallazgo

ID	Evento observado	Frames / valores medidos	Hipótesis	Clasificación	Verificado (n.º de pruebas)
E-xx					

D.2 Ficha de dirección de RAM

Dirección	Escrita por	Leída por	Función	Confianza	Evidencia
`$xxxx`					

D.3 Ficha de WING

Campo	Contenido
WING	
Valor de `$00CF`	
Proyectil	
Cantidad	
Velocidad	
Dirección	
Alcance	
Duración	
Cadencia	
Impacto	
Estructuras	
Bosses	
Tras Morph / crash	

D.4 Ficha de enemigo

Campo	Contenido
Enemigo	
Aparición	
Activación	
Trayectoria	
Velocidad / aceleración	
Seguimiento	

Campo	Contenido
Disparos	
Hitbox	
Estados	
Retirada	
Puntos	